

JAVA INTERVIEW PREPARATION

CHAPTER 10

OPTIONAL AND
NULL-SAFETY DESIGN

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Optional and Null-Safety Design: Making Absence Explicit

Senior / Lead Java Developer Reading Chapter

Null is useful because it can represent "no value," but the Java type system normally does not tell a caller whether null is permitted, expected, or accidental. A method returning `Customer` could return a real customer, return null because no record exists, or fail because a collaborator unexpectedly returned null. Those states look identical at the call site until documentation, tests, or a production failure reveals the difference.

`Optional<T>` gives a method a type-level vocabulary for an expected value that may be absent. It does not eliminate null from Java, and it is not a universal replacement for every nullable field, parameter, collection element, or persistence column. Its value lies in forcing the API author and caller to acknowledge absence and choose what it means.

Senior-level design starts by separating expected absence from invalid state and operational failure. Optional addresses the first category. Validation, invariants, exceptions, and explicit result types address the others.

Absence, Invalid State, and Failure Are Different

Consider a customer lookup. Several outcomes are possible:

```
customer exists      -> Customer
customer does not exist -> expected absence
customer id is invalid -> caller or validation error
database is unavailable -> operational failure
mapped row is corrupt -> invariant or data error
```

Only expected absence naturally maps to `Optional.empty()`.

```
Optional<Customer> findById(CustomerId id) {
    return customerRepository.findById(id);
}
```

A database timeout should not become empty:

```
Optional<Customer> findById(CustomerId id) {
    try {
        return customerRepository.findById(id);
    } catch (DatabaseException exception) {
        throw new CustomerLookupException(id, exception);
    }
}
```

If failure is converted to empty, downstream code may report "customer not found," return HTTP 404, create a duplicate record, or cache the false absence. The API becomes unable to distinguish business state from infrastructure failure.

This is the first Optional rule: emptiness must have one clear domain meaning.

Creating Optional Values Correctly

The three basic factories have different contracts:

```
Optional<Customer> known = Optional.of(customer);
Optional<Customer> possiblyMissing = Optional.ofNullable(databaseResult);
Optional<Customer> missing = Optional.empty();
```

`Optional.of(value)` rejects null immediately with `NullPointerException`. Use it when null would violate an invariant. That fail-fast behavior identifies the contract breach close to its source.

`Optional.ofNullable(value)` converts null to empty. Use it at a boundary where a legacy or third-party API legitimately represents absence with null.

```
Optional<String> environmentValue(String name) {
    return Optional.ofNullable(System.getenv(name));
}
```

Do not wrap arbitrary suspicious values with `ofNullable` merely to prevent an exception. It can hide a bug by reclassifying unexpected null as normal absence.

An `Optional` reference itself must never be null:

```
// Broken contract: callers now have two forms of absence.
Optional<Customer> findCustomer(CustomerId id) {
    return null;
}
```

Return `Optional.empty()`. Otherwise callers may encounter a `NullPointerException` while trying to use the very type intended to model absence safely.

Transforming Values with map

`map` transforms the contained value when present. If the `Optional` is empty, the function is not invoked and the result remains empty.

```
Optional<String> email = customerRepository.findById(id)
    .map(Customer::emailAddress)
    .map(EmailAddress::value);
```

```
Optional<Customer>
|
|-- present --> Customer::emailAddress --> EmailAddress::value
|                                     --> Optional<String>
|
|-- empty -----> Optional.empty()
```

The mapper should normally return a non-null value. `Optional.map` converts a null mapper result to empty, but relying on that behavior can hide a broken domain invariant.

```
Optional<String> email = customer.map(Customer::email); // null becomes empty
```

If `Customer.email()` is documented as non-null, an unexpected null should probably fail validation rather than silently look like an absent customer email. Establish the null contract at the domain boundary.

Use `filter` to retain a present value only when it satisfies a condition:

```
Optional<Customer> activeCustomer = customerRepository.findById(id)
    .filter(Customer::isActive);
```

This is appropriate only if inactive and missing should intentionally produce the same outcome. If the caller must distinguish "not found" from "found but inactive," use explicit branching or a richer result type.

Flattening Nested Optional Values

Use `flatMap` when the transformation already returns `Optional`.

```
Optional<Address> shippingAddress = customerRepository.findById(customerId)
    .flatMap(Customer::preferredShippingAddress);
```

Using `map` would create a nested type:

```
Optional<Optional<Address>> nested = customerRepository.findById(customerId)
    .map(Customer::preferredShippingAddress);
```

```
map:      T -> Optional<R>   produces Optional<Optional<R>>
flatMap:  T -> Optional<R>   produces Optional<R>
```

`flatMap` expresses a sequence of dependent lookups without nesting:

```
Optional<Currency> billingCurrency = customerRepository.findById(customerId)
    .flatMap(Customer::billingAccountId)
    .flatMap(accountRepository::findById)
    .map(BillingAccount::currency);
```

The chain stops at the first empty stage. This is convenient, but it also removes information about which lookup was empty. If the distinction matters for error messages, metrics, or recovery, name the steps and handle their absence separately.

Unlike `map`, the mapper passed to `flatMap` must return a non-null `Optional`. Returning null is a contract violation and causes a `NullPointerException`.

Consuming an Optional

`Optional` supports several ways to define behavior at the boundary.

Return a default value:

```
String displayName = customer
    .map(Customer::displayName)
    .orElse("Guest");
```

Create a fallback lazily:

```
Customer customer = customerRepository.findById(id)
    .orElseGet(() -> guestCustomerFactory.create(id));
```

Fail with domain context:

```
Customer customer = customerRepository.findById(id)
    .orElseThrow(() -> new CustomerNotFoundException(id));
```

Perform one of two actions:

```
customer.ifPresentOrElse(
    auditService::recordAccess,
    () -> auditService.recordMissing(id)
);
```

Supply another `Optional` lazily with `or`, available since Java 9:

```
Optional<Customer> customer = primaryRepository.findById(id)
    .or(() -> archiveRepository.findById(id));
```

`ifPresent` is suitable for optional side effects, but using it to assign an outer mutable variable usually indicates awkward control flow. Prefer returning, mapping, or resolving the `Optional` at a clear boundary.

orElse and orElseGet: Eager versus Lazy

The difference is evaluation timing, not only performance.

```
Customer customer = repository.findById(id)
    .orElse(createGuestCustomer(id));
```

`createGuestCustomer(id)` is evaluated before `orElse` is called, even when the `Optional` contains a `Customer`.

```
Customer customer = repository.findById(id)
    .orElseGet(() -> createGuestCustomer(id));
```

The supplier runs only when the `Optional` is empty.

	Optional present	Optional empty
<code>orElse(value)</code>	fallback evaluated value discarded	fallback evaluated value returned
<code>orElseGet(supplier)</code>	supplier not called	supplier called

This can be a correctness issue when fallback creation has side effects:

```
Order existing = orderRepository.findById(requestId)
    .orElse(createAndPersistOrder(request)); // may persist unnecessarily
```

The eager fallback could insert a duplicate even though an existing `Order` is returned. Use `orElseGet` for expensive, stateful, blocking, or allocation-heavy fallback work.

Use `orElse` for cheap constants or already-computed values:

```
String locale = configuredLocale.orElse("en-US");
```

Avoid the isPresent Plus get Pattern

This code treats `Optional` like a verbose null check:

```
if (customer.isPresent()) {
    sendWelcomeEmail(customer.get());
}
```

Prefer:

```
customer.ifPresent(this::sendWelcomeEmail);
```

For transformation:

```
String name;
if (customer.isPresent()) {
    name = customer.get().displayName();
} else {
    name = "Guest";
}
```

Becomes:

```
String name = customer
    .map(Customer::displayName)
    .orElse("Guest");
```

`get()` without a presence check throws `NoSuchElementException` and communicates no domain meaning. Prefer `orElseThrow` with a meaningful exception when presence is required.

Branching is not forbidden. If present and absent paths involve several coordinated actions, an ordinary `if` can be clearer. `isPresent` or `isEmpty` is reasonable when it supports readable control flow instead of immediately calling `get`.

Optional as a Return Type

Optional is strongest as the return type of a lookup or calculation where absence is expected and the caller must choose a policy.

```
interface PromotionRepository {
    Optional<Promotion> findApplicablePromotion(Customer customer, Cart cart);
}
```

The signature tells callers that no applicable promotion is a normal outcome.

Do not return Optional when absence is impossible by contract:

```
// A User must always have an id after construction.
UserId id();
```

Returning `Optional<UserId>` would weaken the invariant and force every caller to handle a state the object should prevent.

Also consider whether absence should be a named domain result:

```
sealed interface ReservationResult {
    record Confirmed(Reservation reservation) implements ReservationResult {}
    record WaitListed(Position position) implements ReservationResult {}
    record Rejected(RejectionReason reason) implements ReservationResult {}
}
```

Optional cannot explain why there is no reservation. A sealed hierarchy carries alternatives and data explicitly.

Optional Parameters Usually Make APIs Harder

A parameter such as `Optional<DiscountCode>` forces the caller to construct a wrapper and leaves open the possibility that someone passes null instead of `Optional.empty`.

```
Money calculatePrice(Cart cart, Optional<DiscountCode> discountCode)
```

Often clearer designs are overloads:

```

Money calculatePrice(Cart cart) {
    return calculatePrice(cart, Discount.none());
}

Money calculatePrice(Cart cart, Discount discount) {
    // discount is a non-null domain value
}

```

Or a request object with explicit optional properties and validation:

```

record PricingRequest(
    Cart cart,
    DiscountCode discountCode
) {
    PricingRequest {
        Objects.requireNonNull(cart);
    }
}

```

Whether the field may be null should be stated with project nullability annotations or a domain wrapper. Optional parameters can still be reasonable in internal functional composition, but they are rarely the clearest public API.

Optional Fields and Domain Models

Storing Optional in a field is legal, but it introduces integration and modeling questions.

```

final class CustomerProfile {
    private final Optional<PhoneNumber> phoneNumber;
}

```

Potential problems include:

- [Optional](#) is a value-based class and should not be used for identity-sensitive operations.
- [Optional](#) does not implement [Serializable](#).
- Persistence providers generally expect entity attributes they know how to map.
- Serialization libraries may require modules or produce an unexpected JSON shape.
- Framework reflection, bean conventions, and code generation may not expect Optional fields.
- The object can still be corrupted if the Optional reference itself becomes null.

A common domain design stores a nullable private value but exposes Optional at the access boundary:

```

final class CustomerProfile {
    private final PhoneNumber phoneNumber;

    Optional<PhoneNumber> phoneNumber() {
        return Optional.ofNullable(phoneNumber);
    }
}

```

This is not automatically superior; it deliberately keeps null inside one encapsulated implementation and gives callers an explicit contract. For an immutable domain model, a named type such as [ContactPreference](#), [PhoneContact](#), or [NoPhoneContact](#) may model the state more richly.

Avoid setters that accept Optional merely to mirror an Optional-returning getter. The object's internal invariant should define whether the value may be absent, while the public methods should use the clearest type for each operation. Getter and setter signatures do not need to be mechanically symmetrical when they represent different responsibilities.

JPA Entities, Repositories, and Lazy Associations

Repository lookup methods commonly return Optional:

```
Optional<CustomerEntity> findByExternalId(String externalId);
```

This expresses that no row may match. It does not mean entity fields should be declared as Optional. JPA mappings, proxies, dirty tracking, and query derivation are designed around mapped scalar values and associations rather than Optional-valued attributes.

Do not use Optional to hide a lazy-loading boundary:

```
Optional<AddressEntity> address = Optional.ofNullable(customer.getAddress());
```

Calling `getAddress()` may initialize a proxy, issue a query, or fail outside the persistence context. Wrapping the result does not change those lifecycle semantics.

Resolve entities within a deliberate transaction, fetch the data required by the use case, and map to an application or API representation. Optional communicates row absence; it does not manage persistence context ownership.

DTOs, JSON, and API Compatibility

An Optional field in a REST DTO does not automatically produce the desired JSON.

Clients may need to distinguish:

```
property omitted      -> client did not provide an instruction
property explicitly null -> client requests clearing the value
property has a value  -> client requests replacement
```

This three-state PATCH contract cannot be represented by a simple present-or-empty Optional without serializer-specific conventions. Some deserializers map both omitted and null to empty, destroying the distinction.

Use an explicit patch-field type, JSON merge-patch semantics, presence tracking, or a request model designed for the protocol.

For response DTOs, decide whether absence is represented by an omitted property, JSON null, an empty collection, or another documented form. Configure and test the serializer rather than assuming the in-memory Optional shape defines the wire contract.

API contracts outlive implementation choices. Avoid exposing a Java-specific wrapper in a way that makes non-Java clients reason about Optional internals.

Collections and Optional

An empty collection normally represents "there are no elements" more naturally than `Optional<List<T>>`.

```
List<Order> findOrders(CustomerId customerId) {
    return orderRepository.findAllByCustomerId(customerId);
}
```

Returning an empty List allows callers to iterate, stream, or count without an extra absence layer.

`Optional<List<Order>>` may be meaningful only when two states differ:

- The customer does not exist.

- The customer exists but has no orders.

Even then, a result object is often clearer:

```
sealed interface CustomerOrders {  
    record Found(List<Order> orders) implements CustomerOrders {}  
    record CustomerMissing(CustomerId id) implements CustomerOrders {}  
}
```

A collection of Optional values is also often a smell:

```
List<Optional<Customer>> customers;
```

If positions must align with an input batch, the empties may be meaningful. Otherwise, store present values and separately record failures or missing identifiers.

Optional and Streams

`Optional.stream()`, available since Java 9, produces a zero-or-one-element Stream.

```
List<Customer> customers = ids.stream()  
    .map(repository::findById)  
    .flatMap(Optional::stream)  
    .toList();
```

This is concise when missing records should be omitted. If omissions must be investigated, the pipeline loses useful identifiers.

```
record LookupResult(CustomerId id, Optional<Customer> customer) {}  
  
List<LookupResult> results = ids.stream()  
    .map(id -> new LookupResult(id, repository.findById(id)))  
    .toList();
```

Now the caller can separate found and missing IDs and report both.

You can also stream a single Optional to compose with Stream operations, but do not turn a straightforward `map` into a pipeline merely for stylistic consistency. Choose the smallest abstraction that remains clear.

Primitive Optional Types

`OptionalInt`, `OptionalLong`, and `OptionalDouble` avoid boxing for primitive results.

```
OptionalInt maximumRetryCount = configurations.stream()  
    .mapToInt(Configuration::retryCount)  
    .max();
```

They have specialized accessors such as `getAsInt` and fewer transformation methods than `Optional<T>`. Use them naturally at primitive-stream terminals or measured numeric hot paths.

Do not contort domain APIs solely to avoid boxing. An `Optional<Money>` or `Optional<CustomerId>` communicates a real domain type that primitive alternatives cannot replace.

Optional Is a Value-Based Class

Optional instances should be treated solely by their contents. Do not depend on object identity, use an Optional as a synchronization lock, or assume `Optional.empty()` must be the same instance everywhere.

```
// Wrong: identity is not part of the Optional contract.
if (result == Optional.empty()) { }
```

Use content operations such as `isEmpty`, `isPresent`, `equals`, `map`, and `orElse`.

The current runtime may cache empty instances, but implementation reuse is not an application guarantee. Identity-based code is brittle and violates the value-based contract.

Null-Safety Requires More Than Optional

Optional helps at selected return boundaries. A broader null-safety strategy also includes:

- Constructor and method precondition validation.
- Immutable objects that cannot enter partially initialized states.
- Nullability annotations understood by IDEs and static-analysis tools.
- Empty collections instead of null collections.
- Database **NOT NULL** constraints aligned with domain invariants.
- Bean Validation or equivalent checks at external input boundaries.
- Explicit mapping from wire models to validated domain models.
- Tests for nullable integration points.

```
record CreateCustomerCommand(String name, EmailAddress email) {
    CreateCustomerCommand {
        Objects.requireNonNull(name, "name");
        Objects.requireNonNull(email, "email");
        if (name.isBlank()) {
            throw new IllegalArgumentException("name must not be blank");
        }
    }
}
```

The record establishes non-null invariants at construction. Returning Optional from every accessor would weaken that design.

Different annotations such as `@Nullable` and `@NonNull` have framework-specific and tool-specific semantics. Standardize one approach across the codebase and enforce it in builds where possible. An annotation without analysis or team agreement is only documentation.

Caching Absence and Concurrency

Caching Optional can represent both positive and negative cache entries:

```
Cache<CustomerId, Optional<Customer>> cache;
```

This may prevent repeated database work for missing IDs, but negative caching needs its own expiry policy. A Customer created after an empty result could remain invisible until the negative entry expires.

Also distinguish "not cached" from "cached as absent." A Map returning null from `get` can mean no mapping, while a stored `Optional.empty` means a deliberate negative result.

Under concurrency, do not implement check-then-load with separate operations:

```
Optional<Customer> cached = cache.get(id);
if (cached == null) {
    cached = repository.findById(id);
    cache.put(id, cached);
}
```

Several threads can perform the same load. Use the cache library's atomic loading facility or a correct `computeIfAbsent` design, while considering failure caching, recursion, blocking, and stampede control.

Optional models the cached value state; it does not provide atomicity or freshness.

Performance and Allocation

Optional is a small wrapper, but creating large numbers of wrappers can contribute allocation in hot paths. The JVM may eliminate some short-lived allocations through optimization, but application correctness must not depend on that.

Do not replace a clear Optional API with null solely because of hypothetical allocation. Measure using representative workloads, allocation profiling, and end-to-end latency.

Conversely, avoid putting Optional into every field and parameter under the assumption that explicitness always helps. Large object graphs, serializers, ORM integration, and frequently invoked low-level APIs may pay both complexity and allocation costs without improving the contract.

At service boundaries, database calls, network requests, serialization, and logging usually dominate Optional construction. Optimize the expensive workflow before micro-optimizing the wrapper.

Testing Optional Contracts

Test both present and empty outcomes, but also test what emptiness means.

```
@Test
void returnsEmptyWhenCustomerDoesNotExist() {
    when(repository.findById(customerId)).thenReturn(Optional.empty());

    Optional<Customer> result = service.findCustomer(customerId);

    assertTrue(result.isEmpty());
}
```

Test that infrastructure failure remains failure:

```
@Test
void doesNotTranslateDatabaseFailureIntoMissingCustomer() {
    when(repository.findById(customerId))
        .thenReturn(new DatabaseUnavailableException());

    assertThrows(
        CustomerLookupException.class,
        () -> service.findCustomer(customerId)
    );
}
```

Also verify:

- Fallback suppliers are invoked only when empty.
- Expensive or stateful fallbacks are not evaluated eagerly.
- Null-returning dependencies fail or translate according to policy.
- JSON omitted, null, and present properties remain distinguishable when required.

- Repository transactions remain active for data needed by the mapping.
- Negative cache entries expire and are invalidated correctly.
- Public methods never return a null Optional reference.

Tests should assert the semantic contract, not only whether an Optional happens to be empty.

Production Perspective

Optional-related production failures usually come from collapsed states or misplaced boundaries. A database timeout is converted to empty and returned as 404. `orElse` eagerly inserts a fallback record on every successful lookup. A PATCH endpoint treats omitted and explicit-null properties identically and clears customer data. A JPA entity exposes Optional fields that the persistence provider cannot map. A stream silently drops missing IDs. A long negative-cache TTL hides newly created records. Legacy code tries to mutate a fallback object that was intended to be shared.

When investigating Optional behavior, ask:

- What exact state does empty represent?
- Can the operation fail, and how is failure kept distinct from absence?
- Is a fallback eager or lazy, pure or side-effecting?
- Does a map or flatMap stage hide which lookup was empty?
- Is Optional crossing a persistence, serialization, or remote API boundary?
- Are null, omitted, empty, and invalid states being collapsed?
- Is an empty collection more appropriate?
- Does the caller require a reason rather than simple absence?
- Is negative absence cached, and how does it become fresh again?

The solution is often not "use more Optional." It is to model each meaningful state once, at the correct boundary, and prevent impossible states from entering the domain.

Interview-Ready Summary

Optional represents an expected value that may be absent. `of` enforces non-null presence, `ofNullable` adapts a legitimate nullable boundary, and `empty` represents absence. An Optional-returning method must never return null.

`map` transforms a present value and preserves emptiness. `flatMap` is used when the transformation already returns Optional and prevents nesting. `filter` converts a present value that fails a predicate into empty, so it should be used only when that loss of distinction is intentional.

`orElse` evaluates its argument eagerly, while `orElseGet` invokes a supplier only for an empty Optional. This difference affects correctness when fallback work performs I/O, mutation, allocation, or persistence.

`orElseThrow` should attach domain context when absence violates the caller's requirement.

Optional is most effective as a return type for expected absence. It is usually weaker as a public parameter, entity attribute, DTO field, or collection wrapper because frameworks, serialization, and protocol semantics may require different representations. Empty collections usually express "no elements" better than Optional collections.

Absence is not the same as invalid input or operational failure. Do not convert timeouts, corrupted data, authorization failure, or dependency errors into `Optional.empty`. When callers need reasons or multiple alternatives, use a domain result type such as a sealed hierarchy.

Optional is value-based: do not rely on identity, locking, or serialization. It does not provide transaction management, cache freshness, concurrency control, or complete null safety. Combine it with validated constructors, nullability analysis, database constraints, immutable models, explicit wire contracts, and meaningful failure handling.

Revision Notes

- Optional models expected absence, not every possible failure.
- Empty must have one clear domain meaning.
- Do not convert database, network, authorization, or data failures into empty.
- `Optional.of` rejects null and is appropriate for required values.
- `Optional.ofNullable` adapts a legitimately nullable boundary.
- An Optional reference itself must never be null.
- `map` transforms a present value and preserves emptiness.
- A null result from `map` becomes empty and can hide a broken invariant.
- `flatMap` prevents `Optional<Optional<T>>` when the mapper returns Optional.
- A flatMap mapper must not return null.
- Use `filter` only when rejected and absent values may share the same outcome.
- `orElse` evaluates its fallback eagerly.
- `orElseGet` evaluates its supplier only when the Optional is empty.
- Eager fallback evaluation can cause unnecessary I/O or unintended side effects.
- Prefer `orElseThrow` with domain context when a value is required.
- Avoid using `get` as the normal consumption mechanism.
- `isPresent` followed immediately by `get` usually misses Optional's composition API.
- Ordinary branching remains valid when it expresses complex alternatives more clearly.
- Optional is strongest as a lookup or calculation return type.
- Do not use Optional where object invariants guarantee presence.
- Use a sealed result or domain type when absence needs a reason.
- Optional parameters are usually less clear than overloads or validated request objects.
- Optional fields require deliberate persistence and serialization decisions.
- Optional is not Serializable and should not be used for identity-sensitive operations.
- JPA repository methods may return Optional, but entity attributes generally should not use it.
- Wrapping a lazy association does not change transaction or proxy behavior.
- JSON omitted, explicit null, and present are three distinct protocol states.
- Optional alone may be insufficient for PATCH semantics.
- Return empty collections rather than Optional collections when no elements is the only empty state.
- A collection of Optional values needs a positional or diagnostic reason to exist.
- `Optional.stream` is useful when dropping absence is intentional.
- Preserve identifiers when missing batch results must be reported.
- Primitive Optional types avoid boxing for primitive results.
- Do not compare Optional objects by identity or use them as locks.
- Null safety also requires validation, invariants, annotations, constraints, and tests.
- Standardize nullability annotations and enforce them with tooling.
- Negative caching needs expiry and invalidation distinct from positive caching.
- Optional does not make cache loading atomic or prevent stampedes.
- Measure allocation before optimizing Optional away from clear service contracts.
- Test present, empty, failure, fallback timing, serialization, and caching semantics.
- Diagnose Optional incidents by identifying which distinct states were collapsed.